

Smart Vibration Sensor

Spring 2026 ECE 56800 Project Final Report

Vibe Coders

Simon Horning (Lead), Kyle Balon, Kyle Coutee, Kristian Debski, Braydon Fox, Mandy Gu,
Dhirondaatto Sarkar

ABSTRACT

Modern vibration monitoring systems are often expensive, power-intensive, and locked into proprietary cloud ecosystems, making them impractical for cost- and power-constrained deployments. We propose a low-cost, power-efficient smart vibration sensor built on the ESP32 microcontroller and MPU-6050 sensor, with a total hardware costs of \$35. The system periodically wakes from a low-power state to acquire acceleration data, perform on-device signal processing including filtering and Fast Fourier Transform (FFT)-based spectral analysis, and transmit data visualizations over Wi-Fi. Targeted alerts due to anomaly detection are also reported. System applications include predictive maintenance for low-frequency industrial machinery and structural health monitoring of public infrastructure. We evaluate the system on the basis of cost, power consumption, and detection accuracy, using controlled test inputs from a signal generator and real-world validation on household appliances.

1 INTRODUCTION

The collection and profiling of vibration data is an essential process for safety critical monitoring of public infrastructure and factory machinery. As an example, significant funds are spent every year to inspect bridges [2]. Additionally, many companies frequently inspect their factory equipment by performing regular vibration testing [7]. The limitation of both of these methods is that they require the transport, setup, and teardown of inspection equipment as well as a significant investment in manual labor to conduct, verify, and report on results. Internet of Things (IoT) vibration monitoring is available from many sources to address these problems, but these solutions are high-cost [3] and often require customers to integrate into the provider's product ecosystem [4]. Because of this, data processing is handled off-chip in the cloud. An embedded IoT solution from Vibe Coders, a group of masters and PhD graduate engineering students with vibration, embedded system, and signal processing experience, offers a low cost, low barrier to entry system with on-chip processing, custom alerts, and power analysis.

2 BACKGROUND

While there are quite a few commercial options available for vibration sensing, many of them have niche applications, prohibitive cost barriers, and compartmentalized systems that require additional hardware or software to be purchased for effective monitoring. Many lower cost vibration sensors require special mounting hardware or are designed for specific machinery/equipment [1], restricting their uses to only a handful of situations. Even with lower cost devices their prices can still reach upwards of a few hundred dollars [6] while the more expensive systems easily break

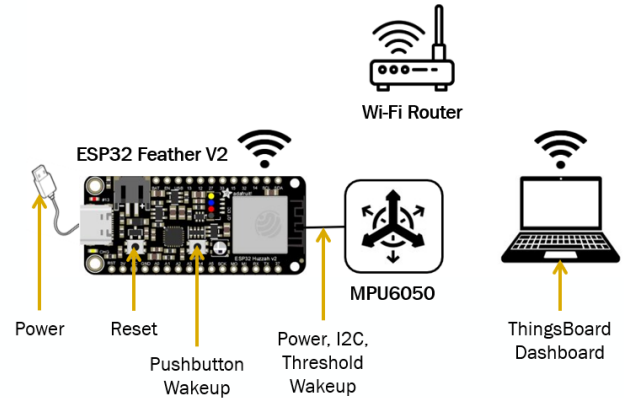


Figure 1: Smart Vibration Sensor Hardware Overview

into the thousands [3]. In addition to this, not all sensors will provide built-in software to simplify the control and monitoring of the device, especially not the simpler systems [1]. They often require you to already have a third party monitoring software or offer one to purchase in addition to the cost of the sensor itself. Therefore, our goal is to package an "all-in-one" device which offers broad application, low cost, and built in monitoring software.

3 PROBLEM STATEMENT

The expansion of IoT technology has enabled connected solutions for civil structure monitoring and smart factories, yet commercially available systems remain costly, proprietary, and not optimized for low-power operation. As a result, many scenarios that could benefit from a connected solution remain unmonitored.

Additionally, contemporary vibration monitoring systems frequently depend on expensive, high-precision piezoelectric accelerometers built to capture a broad frequency spectrum up to the kHz range for detailed fault diagnostics, and they typically rely on continuous data acquisition and transmission for cloud-based analysis, making them power-intensive and complex to deploy. Consequently, they are often over-engineered for applications with tight cost and power constraints that require only basic anomaly detection or low-frequency vibration monitoring.

4 PROPOSED APPROACH

We present the Smart Vibration Sensor: a low-cost, power-efficient, edge-based device designed for easy deployment at scale. As shown in Figure 1, the design is built around the Adafruit ESP32 Feather V2 and MPU-6050 IMU, keeping total hardware cost under \$50. Existing solutions typically rely on expensive proprietary components and

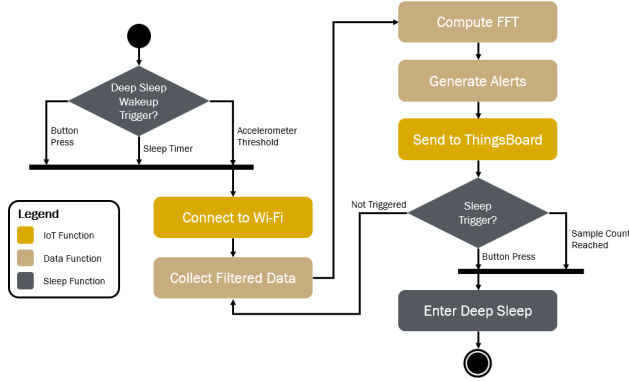


Figure 2: Software Flowchart

subscription-based cloud connectivity — our design eliminates both, using off-the-shelf parts without sacrificing capability.

The Smart Vibration Sensor performs signal processing and data analysis directly on-chip — filtering accelerometer data, computing FFTs, and triggering configurable alerts — before transmitting only the relevant results to the cloud. This edge-processing approach also makes the sensor well-suited for power-constrained environments, where its sleep strategy minimizes energy draw without sacrificing responsiveness. The following subsections detail each feature or function in more detail.

4.1 Filter

The MPU-6050 accelerometer includes an onboard digital low pass filter. However, preliminary results from testing the accelerometer showed that the onboard filter provided insufficient smoothing (see the blue curve in Figure 3 for the accelerometer’s raw x-axis output data).

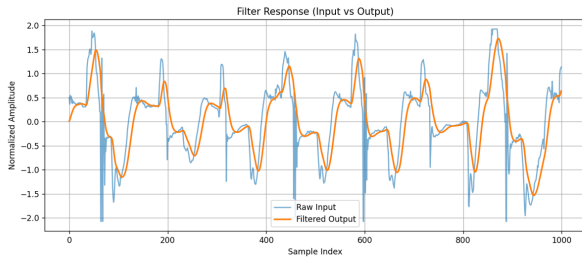


Figure 3: Raw and Filtered Accelerometer X-Axis Data using Second-Order EMA Filter with 31 Hz Cutoff

Our team sought to further attenuate high-frequency components to better isolate and visualize the spectral content of interest. To accomplish this, an additional digital low-pass filter was added to process the incoming accelerometer data. Multiple filter architectures were explored and evaluated, including an exponential moving average filter (EMA), first through third order, and a Butterworth IIR, second and fourth order (see Figures 4a and 4b for the second-order EMA and Butterworth frequency responses, respectively).

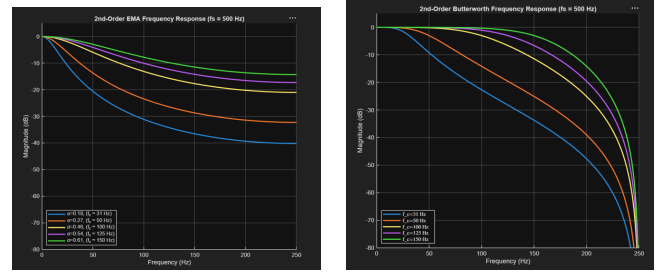


Figure 4: Second-order EMA and Butterworth low-pass filter frequency responses.

Ultimately, the second-order EMA filter was selected because its frequency response satisfied our application requirements while also providing low computational overhead and ease of implementation. The approximate frequency cutoff of the EMA is tunable via a compile-time parameter alpha. By adjusting this parameter, the filter cutoff frequency can be varied from approximately 1 Hz to 150 Hz, depending on the anticipated frequencies of interest. The filtered accelerometer output using a cutoff frequency of 31 Hz is shown by the orange curve in Figure 3.

4.2 FFT

Before the FFT can be computed, it is necessary to store the information in the data stream in an array or buffer so that the computation can be carried out on the data contained in the buffer. Although there exist adaptations of the FFT algorithm which can work on a sample-by-sample basis (see [5]) and track certain frequency components well, it seemed reasonable to stick with the general short-term fourier transform algorithm.

In order to ensure that the data collection can be carried out during FFT computation, without corrupting the data, we implemented a ping-pong buffering system (figure 5), where 2 buffers exist for each x,y,z data from the accelerometer. Once a buffer is full, we swap buffers and flag that the data is ready while indicating the buffer that contains the necessary data.

Once the flag is seen to be activated in the main loop, we compute the FFT and RMS values for the oscillation data. The oscillation data is defined for a set of data X as $X_{osc} = X - \bar{X}$, where $\bar{X}$ is the mean of the data. The computation is carried out using the ulab library for micropython, which implements efficient versions of the algorithm, suitable to be run easily on the ESP32. In Figure 6, we show some execution times for computing the FFT over some size of data set 2^N . Since the FFT algorithm gives us complex numbers, there are 2 ways to get the magnitudes: (1) compute the FFT and compute the magnitudes manually, (2) use the in-build `utils.spectrogram()` function which gives the magnitudes directly. We compare the execution times of these 2 methods in Figure 6, with (2) being the clearly better alternative. With computation on 3 sets of data and a data size of 512 (with a sampling rate of 500 Hz, this amounts to 1s of data), along with a Hanning window for correction, we expect a computation time of close to 25 ms. The expression for the hanning

window is given by

$$w_{\text{hanning}}(n) = \frac{1}{2} \left[1 - \cos \left(\frac{2\pi n}{N} \right) \right]$$

where n is the sample index for the data set $X = [X_n]$. The coherent gain for the window is also used to correct the FFT magnitudes after computation.

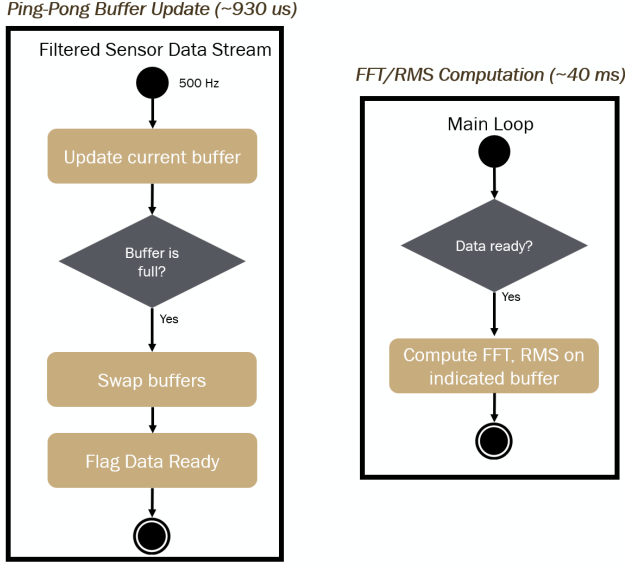


Figure 5: FFT Buffering and Computation Subsystem using ping-pong buffers

We also process the magnitude data to find peak frequencies and amplitudes, which is used later. The oscillation rms is also computed as

$$A_{rms} = \sqrt{X_{osc,rms}^2 + Y_{osc,rms}^2 + Z_{osc,rms}^2}$$

where $X_{osc,rms}$, $Y_{osc,rms}$, $Z_{osc,rms}$ are found similar to what is shown below:

$$X_{osc,rms} = \sqrt{\frac{1}{N} \sum_{i=1}^N X_{osc,i}^2}$$

In its entirety, the computation and processing step takes close to 40 ms, which is much lower than the ping-pong buffering window of 1s. However, it is also necessary to note that the current setup gives a latency of close to 1s for real time frequency detection, which is enough for this project. This can be sped up by more careful selection of the sample length, which also affects the frequency resolution of the FFT. This is an important trade-off to consider in real systems.

4.3 Low Power

For potential use cases where power consumption is a concern, a power management strategy has been considered which focuses on edge computing, deep sleep utilization, and efficient communication.

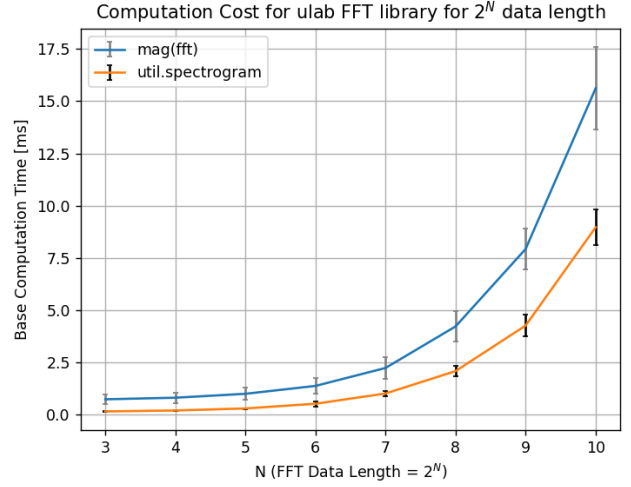


Figure 6: ulab FFT Computation Time for different data lengths

Our greatest contributor to energy saving is the utilization of the ESP32's deep sleep mode. In its default mode, the ESP32 wakes up from deep sleep every 30 seconds to collect sample vibration data. It performs FFT computations as described previously, and sends a predefined number of data points to ThingsBoard over Wi-Fi, then returns to sleep for 30 seconds. Due to variance in connection speeds, packet loss, or other factors that may affect data transport time, successful delivery of the data to ThingsBoard always occurs before the ESP32 returns to deep sleep. This allows guaranteed data availability, rather than risking data loss by going to sleep after an elapsed time. In order to monitor sudden severe motion events, the MPU-6050's built-in motion threshold interrupt is leveraged to wake up the ESP32 from deep sleep when excessive motion occurs. This balances the capture of potentially important data with energy savings. Data transmission over Wi-Fi is perhaps the most power-intensive activity for the system. Utilizing edge computing by transmitting only the relevant FFT data to an external system yields a shorter data transfer time, allowing the ESP to sleep sooner. Additionally, no local data storage is required on-chip. This eliminates the need to use a less efficient sleep mode that preserves RAM, or an external storage method.

4.4 Dashboard

For user interface we decided to use a popular and versatile platform known as ThingsBoard, commonly used for many IoT devices and applications. The dashboard, which can be seen in Figure 7, consists of 13 widgets, 5 of which were custom coded from scratch, specifically for this system.

The four larger widgets on the left hand side form the user's data visualization center. Starting from the top left, you have a graph which shows real-time vibration amplitude and associated frequency bins. In this same widget the user can also change the time frame of data collection and aggregate the amplitude and frequency bins over the set period to form a histogram, allowing detailed analysis of the collected vibration data.

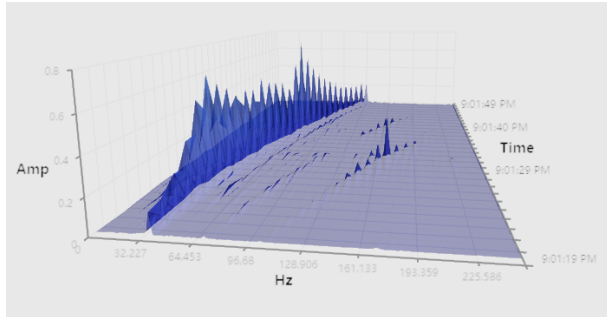
Figure 7: ThingsBoard Dashboard

Custom Calibration Window). The three input fields at the bottom allow the user to define a custom vibration detection range by specifying an amplitude threshold and frequency lower and upper bounds. On the ESP32, each FFT frame is analyzed to obtain the frequency spectrum, and if any frequency within the user-defined range has an amplitude above the specified threshold, the "Vibration Detected" widget updates to display "Y" as an alert to the user, and "N" otherwise. In addition, two widgets continuously display the live peak amplitude and peak frequency extracted from every FFT frame, providing vibration spectral information independent of the detection threshold settings.

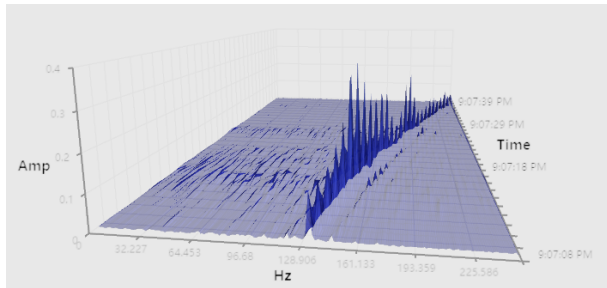
Figure 8: ThingsBoard Rule Chain

Figure 9: Power and Energy Measurement

occurred. Linear frequency sweep results can be seen in Figure 10. This testing method is not without its shortcomings however. Without a calibrated vibration source, it is not possible to evaluate the magnitude accuracy of the system.



(a) Linear Low Frequency Sweep



(b) Linear High Frequency Sweep

Figure 10: Frequency Test Sweeps

6 DISCUSSION

The results show that the Smart Vibration Sensor is viable for its targeted applications. At \$35 in hardware, the system undercuts commercial alternatives while retaining core functionality: on-device FFT, configurable alerting, and a full IoT dashboard. Power measurements confirm the effectiveness of the sleep strategy, with 40 mW being consumed at idle versus 300 mW when active. This indicates extended battery life, but testing with a battery was not addressed in the scope of this project.

The linear frequency sweep results show reliable spectral detection at higher frequencies, with FFT peaks tracking the input stimulus faithfully. Some aliasing was observed at lower frequencies, which is consistent with the MPU-6050's sensitivity characteristics and the interaction between the sampling rate and low-frequency signal content. For the structural and industrial monitoring applications targeted here, this is a manageable limitation, as the frequencies of greatest interest tend to fall in the mid-to-upper portion of the sensor's range.

The two primary limitations of the system are the 1-second FFT buffering latency and the MPU-6050's constrained dynamic range. The 1-second delay is due to the frequency resolution required for meaningful spectral analysis — shorter windows reduce latency but degrade resolution. For slow-moving machinery and structural monitoring, this trade-off is acceptable, but would require revisiting

for faster transient detection. Future work could explore higher-performance IMUs or adaptive sample window sizing to broaden applicability while retaining the low-cost, edge-processing architecture developed here.

The relevant source code for the Smart Vibration Sensor can be found on GitHub [8].

7 POTENTIAL IMPACTS

The Smart Vibration Sensor has several advantages for monitoring low frequency machinery such as low speed fans/pumps, electric motors. A low-cost sensor lowers the barrier to entry for connecting a larger set of monitoring devices while cutting maintenance costs. Additionally, such a sensor would form a low-cost addition to smart homes in order to monitor home appliances, with flexibility in power consumption and real-time monitoring.

8 CHANGE OF PLAN

There were several relatively minor course corrections from the original proposal, mostly related to the low-power strategy, GUI representation, and system alerts. To capture live vibration events, we use an accelerometer threshold interrupt to wake the ESP32 from deep sleep. A button press also triggers device wakeup and is used to calibrate the accelerometer until the button is pressed again. Using ThingsBoard allowed us to reallocate GUI development time to wireframing the ThingsBoard dashboard to represent FFT data, alerts, and take user input. Additionally, the team expanded on system alert features due to additional functionality on ThingsBoard: immediate alerts are handled on-chip (amplitude threshold, amplitude within a user-specified frequency range) and sent to ThingsBoard for display, while time-history alerts are handled using SPC charts on ThingsBoard.

9 EMBEDDED SYSTEMS ASPECTS

This project uses several core concepts in embedded systems. The ESP32 communicates with the MPU-6050 sensor over I2C, a standard embedded communication protocol, and transmits processed data and alerts wirelessly over Wi-Fi. Timers and interrupts are used to schedule periodic data acquisition and trigger alert conditions, which ensures efficient use of the processor. Lastly, the project addresses resource-constrained functionality in both power consumption and computation.

10 TASK BREAKDOWN

The project spanned six weeks of active development, with the primary work streams consisting of: filter design and implementation (Kyle C), FFT computation (Dhironaatto), low-power mode implementation (Braydon), alert system development (Mandy), ThingsBoard data visualization (Kristian), testing and evaluation (Kyle B.), and system and deliverable management (Simon). Table 1 summarizes the task breakdown.

11 RELATION TO OTHER PROJECT

Not applicable.

Table 1: Project Timeline

Week	Tasks
11 (3/23)	Team formation, project proposal, concept of operations, GitHub repository setup
12 (3/30)	Hardware setup and bring-up, I2C communication with MPU-6050, Wi-Fi connection, IoT dashboard trade study, preliminary filter design concepts
13 (4/6)	Low-pass filter design and implementation, FFT investigation, low-power mode investigation, alert brainstorming, ThingsBoard familiarization
14 (4/13)	ThingsBoard bring-up and initial streaming tests, data visualization planning, real-time data transmission and display, alert system implementation
15 (4/20)	Initial system integration of filter and FFT, ThingsBoard development, deep sleep wake-up reason
16 (4/27)	Full system integration, testing and evaluation, bug fixes, remaining dashboard features
16 (5/4)	System finalization, final report and presentation

REFERENCES

- [1] Althen. 2026. Low Frequency (seismic) Vibrations. <https://www.althensensors.com/sensors/vibration-sensors/low-frequency-seismic-vibration-sensors/>
- [2] Andrea Zampieri, Ph.D., Structure Magazine. 2023. Bridge Vibration Monitoring. *Structure Magazine* (2023). <https://www.structuremag.org/article/bridge-vibration-monitoring/>
- [3] enDAQ. 2026. S3 Vibration Sensor. <https://endaq.com/products/s3-vibration-sensor-s3-d40>
- [4] Enersib. 2026. AMS Wireless Vibration Monitor. <https://www.emerson.com/en/automation-systems/asset-performance-management/machinery-health-management/ams-wireless-vibration-monitor>
- [5] E. Jacobsen and R. Lyons. 2003. The sliding DFT. *IEEE Signal Processing Magazine* 20, 2 (2003), 74–80. doi:10.1109/MSP.2003.1184347
- [6] Monnit. 2026. Advanced Vibration Meter. https://www.monnit.com/products/sensors/accelerometers/advanced-vibration-meter/?srsltid=AfmBOor_xKCXTBkNsYFuyOqedzkeUWMeW3u3iHJP-vBysUot536WhAcI
- [7] Trent Phillips, Ludeca. 2023. How Often Do You Collect Vibration Data on Your Equipment? <https://ludeca.com/blog/maintenance-tips/3157/how-often-do-you-collect-vibration-data-on-your-equipment/>
- [8] Vibe Coders. 2026. Smart Vibration Sensor. <https://github.com/Dhirodaatto/Smart-Vibration-Sensor-ECE-568/tree/main>